PALANTIR FOUNDRY PROJECT ENABLEMENT PROGRAM

Day 1 Study Material
Project Architecture & Solution Walkthrough

Phase 1 — Foundation Accelerator

Author: Rajesh Pasham

Table of Contents

1. Session Overview & Objectives

Day 1 opens the Foundation Accelerator phase of the engagement. Before any hands-on development begins, the whole team needs a shared, accurate picture of how Foundry is architected and how the pieces you will build over the next ten days fit together.

By the end of today's session, you will be able to:

- Describe the major architectural layers of the Foundry platform
- Explain the Ontology's core building blocks: object types, properties, link types, and action types
- Navigate the Ontology Manager and locate key resources
- Describe how Workshop, OSDK applications, and Functions consume the Ontology
- Create your first Ontology object types in your own Developer Tier account
- Understand the coding standards and sprint structure for the rest of the engagement

2. Foundry Platform Architecture

Foundry is organized in layers. Each layer builds on the one below it, and every application you build in this engagement touches several of these layers at once. It helps to think of the stack bottom-up: start with data, model it, automate it, then expose it.

2.1 Data Integration (the foundation)

Source systems, files, and databases are connected into Foundry as datasets through Data Connection, and refined through Pipeline Builder or code-based transforms. This is the raw material the rest of the platform operates on. For this project, the UPS and Neology feeds introduced in later sprints will land here before ever reaching the Ontology.

2.2 The Ontology

The Ontology sits above your datasets and turns raw rows and columns into a semantic model of your business: object types, properties, link types, and action types. Palantir describes the Ontology as an operational layer that acts as a digital twin of an organization, connecting digital assets to their real-world counterparts.

2.3 Functions & Automation

Functions hold your business logic — validation rules, calculations, event handlers, and scheduled jobs. Functions read from and write to the Ontology, and are the mechanism by which REST APIs, notifications, and scheduled polling are implemented later in this engagement. Every Action Type you define is ultimately backed by a Function.

2.4 Applications (what people actually use)

Workshop and OSDK-powered applications (React, Python, Java) are how people and other systems interact with the Ontology day to day. Workshop is a low-code builder, well suited to the Inventory, Shipment, and Check-in/Check-out screens built in Phase 2. OSDK applications are custom-coded and give full control over the experience — the path this engagement takes for the Transponder Add/Remove PWA starting Day 3.

3. The Ontology: Core Concepts

An Ontology is a categorization of the world. In Foundry, the Ontology maps datasets and models onto four core concepts:

- **Object Type:** defines an entity or event in your business — for example, Vehicle or Transponder
- **Property:** defines one characteristic of an object type — for example, status, make, or issueDate
- **Link Type:** defines a relationship between two object types — for example, a Vehicle is linked to the Transponder installed on it
- **Action Type:** defines how an object can be created, updated, or deleted — every write to the Ontology goes through an action, never a direct write

3.1 A Familiar Analogy

If you are coming from a data engineering background, this mapping may help:

Dataset World	Ontology World
Dataset	Object Type
Row	Object (an instance of the object type)
Column	Property
Foreign key relationship	Link Type

Preview: Every write to the Ontology happens through an Action Type. We will define our first custom action types in Week 2 (Advanced Ontology Modeling) — today we focus on modeling the objects and their relationships.

4. The Ontology Manager Interface

Ontology Manager is the primary UI for building and browsing the Ontology. You can open it directly by adding /workspace/ontology to the end of your Foundry home page URL.

4.1 Layout

- **Top bar:** search for Ontology resources, create new resources, and navigate or create branches
- **Sidebar:** navigate to different resources, pages, and applications within Ontology Manager
- **Discover view:** a customizable landing page; if you are new to an Ontology, it highlights recently modified and prominent object types

4.2 Object Type View & Property Editor

Selecting an object type opens its detail view, including an Overview page listing its properties. Selecting a property from that list opens the property editor, where you configure its data type, display name, and constraints.

The same object type view also surfaces a Links tab (relationships to other object types) and, once defined, an Actions tab (how the object can be created or modified). You will use all three tabs — Overview, Links, and Actions — across today's and tomorrow's hands-on exercises.

5. Worked Example: Vehicle & Transponder Object Types

These are the two object types you will build by hand in today's hands-on exercise. Walking through their full property lists now makes the hands-on steps faster to follow.

5.1 Vehicle

Property	Type	Notes
vehicleId	String	Primary key / title property
vin	String	17-character VIN
make / model	String	e.g. Ford, Transit-350
year	Integer	
licensePlate	String	
status	String	Active / Maintenance / Inactive
currentLocation	String	Facility name

5.2 Transponder

Property	Type	Notes
transponderId	String	Primary key / title property
serialNumber	String	
status	String	Active / In Stock / Faulty
issueDate	Date	ISO format, e.g. 2023-02-14
deviceType	String	e.g. RFID-Gen2, BLE-Beacon, Hybrid

6. Link Types & Your Data Model

A Link Type is a named, typed relationship between two object types. Today's link — assignedTransponder — connects Vehicle and Transponder:

- **Cardinality:** one-to-one for Day 1 (a vehicle has at most one active transponder)
- **Foreign key:** Transponder.assignedVehicleId equals Vehicle.vehicleId
- **Direction:** assignedTransponder (Vehicle → Transponder) and its reverse, installedOnVehicle (Transponder → Vehicle)

Put together, your Day 1 data model looks like this:

Vehicle	Transponder
vehicleId (PK)	transponderId (PK)
vin	serialNumber
make / model	status
year	issueDate
licensePlate	deviceType
status	assignedVehicleId (FK)
currentLocation	

Why one-to-one for now: Later sprints extend assignedTransponder to one-to-many, so a vehicle's full transponder history can be tracked over time. Modeling it as one-to-one today is a deliberate teaching choice, not a limitation of Foundry.

7. Action Types, Mutability & Governance — A Preview

7.1 Action Types

Action types define how an object can be created, updated, or deleted. Every write to the Ontology goes through an action — never a direct database write. Actions can be triggered from Workshop buttons, APIs, or Functions. We build our first custom action types in Week 2 (Advanced Ontology Modeling).

7.2 Mutability

Foundry objects support append and update patterns; direct deletes are avoided in operational systems in favor of status flags and audit trails. Version history is built on this same append/update model. Week 3 (Mutable Operational Applications) covers this in depth, with a full Inspection Workflow lab that models create/update-only object handling end to end.

7.3 Governance

Governance is enforced at every layer of the platform: object-level permissions control who can see an object type at all, action-level permissions control who can invoke a given action, and branch-based change review controls how Ontology changes reach production. None of this needs to be configured today, but it is worth knowing it is already there, underneath every object and action you create.

8. Workshop — Operational Applications on the Ontology

Workshop is Foundry's low-code application builder. Widgets — tables, forms, maps, charts, buttons — bind directly to Ontology object types and action types, so a working operational tool can be assembled without writing application code.

In this engagement, Workshop is used for the Inventory, Shipment handling, Device location, and Check-in/Check-out screens introduced in Phase 2. Today's hands-on exercise includes an optional first look at building a simple Workshop view.

9. Developer Tooling: OSDK, Developer Console & Foundry CLI

9.1 Ontology SDK (OSDK)

The OSDK is a generated, typed client library (TypeScript, Python, or Java) that gives application code ergonomic, high-scale access to the Ontology — reading and writing objects with minimal boilerplate, while Foundry's governance and permissioning are enforced underneath.

9.2 Developer Console

Developer Console is where OSDK applications and OAuth clients are created and managed. It generates API documentation (TypeScript, Python, and cURL) specific to the object types, action types, and functions your application uses.

9.3 Foundry CLI & Code Repositories

Code Repositories host the source for Functions and applications, with version control and review built in. The Foundry CLI lets you develop locally and publish back to your enrollment.

9.4 SuperRepo

SuperRepo is Foundry's newest application pattern: it lets you define Ontology object types, links, and actions as code in the same repository as your Functions and frontend, with the OSDK regenerated automatically whenever those definitions change. Existing Ontology Manager resources can be imported into a SuperRepo instead of being redefined, so applications are never siloed by how their Ontology types were originally created. This engagement primarily uses the Ontology Manager UI for modeling, with SuperRepo introduced as an advanced option in later weeks.

10. React + OSDK Architecture Preview

Starting Day 2, you will build a React Progressive Web Application that talks to the Ontology through a generated OSDK client:

React PWA → OSDK Client (generated) → Ontology (Objects & Actions)

The OSDK client is generated specifically for your Ontology's object types and actions, so the React application interacts with strongly typed objects such as Vehicle and Transponder rather than raw API calls.

11. Security — A Preview

Role-Based Access Control (RBAC) restricts who can see or act on which object types. Attribute-Based Access Control (ABAC) goes further, restricting access based on data values — for example, a regional manager only seeing assets in their own region.

Week 10 (Enterprise Security) builds a Regional Operations Dashboard that enforces both RBAC and ABAC together. For now, know that every object and action you create already carries a permissions model, even before you configure it explicitly.

12. Coding Standards for This Engagement

- Property naming: lowerCamelCase (e.g. vehicleId, currentLocation)
- Object type display names: PascalCase, singular (e.g. Vehicle, Transponder)
- TypeScript for all React/OSDK application code; Python for Functions and data transforms
- One Code Repository per application; Ontology changes are reviewed before publishing
- Every Pull Request references its Sprint and includes a short description of the Ontology or UI change
- Prefer small, reviewable commits over large batch commits

13. Sprint Plan Recap

Everything built in Phase 1 becomes the foundation for the five guided sprints in Phase 2:

Sprint	Focus
Sprint 1	Ontology — Shipment, Inventory, Transponder, Bin
Sprint 2	React App — Add / Remove Transponder
Sprint 3	Shipment APIs — external integrations, polling
Sprint 4	Barcode, Inventory & Geospatial Visualization
Sprint 5	Testing, Deployment, Security

14. Key Terms Glossary

Term	Definition
Ontology	The semantic layer mapping datasets and models to object types, properties, link types, and action types.
Object Type	Defines an entity or event, e.g. Vehicle.
Property	A characteristic of an object type, e.g. status.
Link Type	A relationship between two object types.
Action Type	Defines how an object can be created, updated, or deleted.
Ontology Manager	The UI for building and browsing the Ontology, at /workspace/ontology.
OSDK	Ontology Software Development Kit — a generated, typed client library for reading and writing Ontology objects from code.
Developer Console	The application used to create and manage OSDK applications and OAuth clients.
Foundry CLI	Command-line tool for local development and publishing to your enrollment.
Code Repository	Version-controlled home for Functions and application source code.
SuperRepo	A repository pattern that defines Ontology, Functions, and frontend together, generating the OSDK from code.
Workshop	Foundry's low-code application builder for operational tools.
Object Storage V2	A modern backing store for Ontology objects, without the 10,000-result paging limit older backends impose.
Data Connection	The Foundry component that brings source systems and files in as datasets.
Pipeline Builder	A low-code tool for cleaning and reshaping datasets before they reach the Ontology.
RBAC / ABAC	Role-Based / Attribute-Based Access Control — the two permission models covered in Week 10.

15. Frequently Asked Questions

- **Do I need coding experience for today?:** No — today is UI-driven in Ontology Manager. Coding starts on Day 2.
- **What if my Developer Tier sign-up isn't available in my region?:** Use the engagement-provisioned Foundry environment instead — every step in the hands-on exercise works identically.
- **Can I use different sample data than the provided vehicles/transponders?:** Yes, but the hands-on steps and troubleshooting notes assume the provided datasets, so following them exactly is the fastest path today.
- **What happens if I make a mistake in Ontology Manager?:** Object types and properties can be edited after creation. If you get stuck, revisit the relevant hands-on step or raise it in Office Hours.

16. Today's Definition of Done

You have completed today's session when:

- Vehicle object type is published, with all 8 properties, backed by real data
- Transponder object type is published, with all 6 properties, backed by real data
- The assignedTransponder link correctly connects at least 10 Vehicle/Transponder pairs
- You can browse both object types and their linked objects in Ontology Manager
- (Stretch) A simple Workshop table view displays your Vehicle objects

17. Reference Documentation

These official Palantir Foundry documentation pages were used as the basis for today's material and are recommended further reading:

- Ontology — Core concepts: palantir.com/docs/foundry/ontology/core-concepts
- Ontology — Overview: palantir.com/docs/foundry/ontology/overview
- Object backend — Overview: palantir.com/docs/foundry/object-backend/overview
- Ontology Manager — Overview: palantir.com/docs/foundry/ontology-manager/overview
- Ontology Manager — Navigation: palantir.com/docs/foundry/ontology-manager/navigation
- Ontology SDK — Overview: palantir.com/docs/foundry/ontology-sdk/overview
- Getting started — Overview: palantir.com/docs/foundry/getting-started/overview
- Foundry announcements (SuperRepo): palantir.com/docs/foundry/announcements/2026-08

Keeping current: Foundry's UI and documentation evolve continuously. Always check the in-platform documentation for the most current screens and terminology before each session.

18. What's Next — Day 2 Preview

Module 1: OSDK Architecture

- OSDK overview and SDK concepts
- Architecture and Object APIs
- Actions and Queries
- Authentication

Hands-on lab: develop your first OSDK application.